Bilkent University
Department of Computer Engineering

Senior Design Project
T2508
Mergent

Analysis and Requirement Report

22103002, Hasan Kutluhan Şıpka, kutluhan.sipka@ug.bilkent.edu.tr
22102787, Erdem Aydın, aerdem@ug.bilkent.edu.tr
22103227, Kaan Aydeniz, kaan.aydeniz@ug.bilkent.edu.tr
22103448, Oğuzhan Genç, oguzhan.genc@ug.bilkent.edu.tr
22102970, Furkan Emre Kolukırık, emre.kolukirik@ug.bilkent.edu.tr

Supervisor: *Eray Tüzün*
Course Instructors: *Mert Bıçakçı, İlker Burak Kurt*

18.12.2025

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfillment of the requirements of the Senior Design Project course CS491/2.

Contents

1 Introduction **3**
2 Proposed System **3**
2.1 Overview **4**
2.2 Functional Requirements **4**
2.2.1. PR-Issue Linkage and Suggestion 4
2.2.2. PR-Issue Alignment Validation 5
2.2.3. AI-Generated Review Comments 5
2.2.4. PR Enrichment with Risk & Impact Insights 6
2.2.5. Vibe Reviewing 6
2.2.6. Post-Review Feedback 7
2.2.7. Organization Monitoring Dashboard 7
2.3 Non-functional Requirements **8**
2.3.1. Usability 8
2.3.2. Reliability 8
2.3.3. Performance 8
2.3.4. Security 9
2.3.5. Availability 9
2.3.6. Scalability 9
2.4 Pseudo Requirements **9**
2.5 System Models **9**
2.5.1 Scenarios 9
2.5.2 Use-Case Model 12
2.5.3 Class Model 13
2.5.4 Dynamic Models 14
2.5.5 User Interface 15
3 Other Analysis Elements **21**
3.1 Consideration of Various Factors in Engineering Design **21**
3.1.1 Constraints 22
3.1.2 Standards 23
3.2 Risks and Alternatives **24**
3.3 Project Plan **26**
3.4 Ensuring Proper Teamwork **36**
3.5 Ethics and Professional Responsibilities **36**
3.6 Planning for New Knowledge and Learning Strategies **36**
Glossary **38**
References **39**

Analysis and Requirement Report

Mergent

1 Introduction

Code reviews are a vital part of modern software development, ensuring code quality, reliability, and maintainability. However, current pull request (PR) reviews are often slow, inconsistent, and unclear. Issue–PR mismatches waste reviewer time, and the median PR approval in large projects can take 15–20 hours [1]. Human review quality varies, with approximately 32–36% of review comments classified as noisy or irrelevant [2]. Furthermore, due to the lack of structured feedback for reviewers, 21.3% of comments are ultimately dismissed as “Won’t Fix” [1]. As a result, review cycles become longer, rework increases, and developers spend 8–16% of their working week on reviews [1].

To overcome these challenges, Mergent is proposed as an AI-powered intelligent code review assistant designed to make reviews faster, clearer, and more consistent. Mergent enhances the review process through six key phases that together form a comprehensive review pipeline. It begins with Issue Linkage and Alignment, where the system detects or generates issue links and ensures each PR truly addresses its intended issue. During the AI Review Comments phase, Mergent instantly provides precise, context-aware, and actionable feedback. In the PR Enrichment stage, it highlights risky modules, flags bug-prone areas with a history of recurring issues, and assigns a simple risk score to guide reviewers’ attention.

The Vibe Reviewing phase introduces a Human-AI partnership, where Mergent collaborates with reviewers to refine vague feedback, maintain consistent tone and focus, and evaluate the quality of reviewer comments to help them improve over time. Finally, the Post-Review Feedback phase delivers a concise summary of the review outcomes and provides clear next-step suggestions for the PR author.

By integrating Large Language Models (LLMs) into a human-in-the-loop workflow, Mergent bridges the gap between human expertise and AI efficiency. The system ultimately aims to reduce review time, improve feedback quality, and establish a more structured and intelligent PR review ecosystem.

2 Proposed System

The Mergent system operates through a structured six-phase pipeline designed to streamline the code review process. It begins with PR-Issue Linkage and Suggestion, where the system automatically identifies or generates links between pull requests and their corresponding issues. This is followed by PR-Issue Alignment Validation, a phase that verifies whether the code changes accurately address the requirements of the linked issue, categorizing the alignment into states

such as "Exact," "Missing," "Tangling", and “Missing and Tangling”. To provide immediate feedback, the AI-generated review comments phase produces context-aware and actionable suggestions directly on the code diff.

The process further enhances reviewer focus through PR Enrichment with Risk & Impact Insights, which flags bug-prone modules and calculates a consolidated risk score based on historical data and production impact. In the Vibe Reviewing phase, the system acts as a collaborator by refining human feedback for clarity and ensuring a consistent professional tone across the development team. Finally, the Post-Review Feedback phase compiles all AI and human insights into a unified summary, offering the PR author clear next steps and quality metrics to conclude the review cycle.

2.1 Overview

Mergent is an AI-powered intelligent code review assistant designed to mitigate the inefficiencies of manual review cycles, which often take 15–20 hours in large-scale projects. By addressing the high volume of noisy or irrelevant feedback, which can account for up to 36% of all review comments, the system aims to streamline the software development lifecycle, making it faster, clearer, and more consistent.

As a comprehensive review solution, Mergent operates through a structured six-phase pipeline: PR-Issue Linkage [5], Alignment Validation [6], AI-Generated Review Comments [1], PR Enrichment with Risk Insights [7], Vibe Reviewing, and Post-Review Feedback. The system employs a multi-agent architecture, where specialized agents collaborate to bridge the "semantic gap" between natural language issue descriptions and technical code changes.

Mergent is built as a human-in-the-loop workflow, where AI acts as a collaborative partner rather than a replacement. It integrates directly with existing development environments, such as GitHub or Bitbucket, and project management tools like Jira to suggest existing issues or generate new ones automatically. By combining Large Language Models (LLMs) with static analysis and historical risk scoring, Mergent provides reviewers with prioritized insights and refined feedback templates, ensuring that code changes are accurately aligned with project requirements and professional standards.

2.2 Functional Requirements

2.2.1. PR-Issue Linkage and Suggestion

The system shall manage the connection between Pull Requests (PRs) and their corresponding issues.

- **Detect Linkage**

The system shall automatically detect whether a submitted PR is linked to an issue.

- **Suggest Existing Issue**

  If a PR is not linked to an issue, the system shall search all existing issues and suggest the most relevant one to be linked.

- **Generate New Issue**

  If a PR is unlinked and no relevant existing issue is found, the system shall be able to automatically generate and link a new issue based on the PR's content and context.

2.2.2. PR-Issue Alignment Validation

The system shall validate that the content of a PR aligns with the requirements of its linked issue.

- **Verify Connection**

  The system shall analyze the code changes in the PR to verify that it correctly addresses the description in the linked issue.

- **Categorize Alignment**

  The system shall classify the alignment into one of the following four categories:

  1. Exact: The PR fully implements the changes described in the issue.
  2. Missing: The PR fails to implement the required changes.
  3. Tangling: The PR includes additional, unrelated changes not mentioned in the issue.
  4. Missing & Tangling: The PR both fails to address the issue's requirements and adds unrelated work.

2.2.3. AI-Generated Review Comments

The system shall be able to automatically generate code review comments.

- **Generate Contextual Comments**

  The system shall generate clear, specific, and actionable AI-powered review comments based on the code changes within the PR.

- **Provide Actionable Suggestions**

The comments should provide concrete suggestions for improvement, such as adding a necessary null check to a function.

2.2.4. PR Enrichment with Risk & Impact Insights

The system shall enrich Pull Requests (PRs) with data that helps reviewers prioritize their focus by analyzing potential risks to the production environment.

- **Identify Impact**

  The system shall highlight the files and modules affected by the changes in the PR.

- **Assess Historical Risk**

  The system shall flag changes in code areas that are historically bug-prone or have been part of past failures.

- **Analyze Production-Level Risk**

  The system shall analyze the potential impact of the PR on the live production environment. This includes flagging changes that could affect critical services, introduce security vulnerabilities, or negatively impact system performance.

- **Provide Risk Score**

  The system shall calculate and display a simple, easy-to-understand risk score that consolidates historical and production-level risk factors for the PR.

2.2.5. Vibe Reviewing

The system shall enable human-AI collaboration to ensure consistent, clear, and high-quality review feedback.

- **Refine Reviewer Feedback**

  The system shall analyze human review comments and rewrite vague or repetitive ones into precise, actionable messages.

- **Ensure Consistency**

  The system shall maintain a uniform tone and review style across all reviewers to avoid conflicting or unclear feedback.

- **Enhance Human Comments**

When a vague comment is detected, the system should suggest a more specific and actionable rewrite to help the reviewer provide more effective feedback.

2.2.6. Post-Review Feedback

The system shall generate a unified post-review summary that consolidates AI and human feedback.

- **Summarize Key Outcomes**

  The system shall compile all review comments into a single, concise summary highlighting major findings and resolutions.

- **Provide Follow-Up Actions**

  The system shall generate clear, prioritized next steps (e.g., add tests, refactor modules, document changes).

- **Deliver Author Insights**

  The system shall provide structured feedback as a report to the PR author that includes improvement areas and overall review quality insights.

2.2.7. Organization Monitoring Dashboard

- **Provide Organizational Overview**

  The system shall provide a monitoring dashboard accessible to team leads for observing repository-level activity and usage of Mergent.

- **Display Organization Productivity Metrics**

  The system shall display productivity-related statistics for pull requests, including the average merge time and the number of merged pull requests over a configurable time period (e.g., last 6 months).

- **Enable Before-and-After Comparison**

  The system shall allow team leads to compare pull request metrics before and after the adoption of Mergent to evaluate its impact on the review process.

- **Show Current Pull Request Status**

The system shall present a dedicated section listing current pull requests along with their associated issues and linkage status.

- **Provide Timeline and Calendar Views**

  The system shall allow team leads to view pull requests and issues in a chronological timeline and calendar-based visualization.

- **Present AI Usage and Cost Insights**

  The system shall display aggregated statistics related to Mergent usage, including the number of AI-generated reviews, total review comments produced, average AI review duration, and average cost per review.

- **Support Section-Based Navigation**

  The system shall organize dashboard content into distinct sections (e.g., productivity metrics, current pull requests, timelines) to enable intuitive navigation.

2.3 Non-functional Requirements

2.3.1. Usability

- Shall integrate seamlessly with one version control system, such as GitHub or BitBucket.
- AI feedback and scores should be clear and easy to understand.

2.3.2. Reliability

- AI suggestions and alignment checks should be accurate and trustworthy.
- Analysis should be consistent for similar PRs to ensure predictable results.
- Shall gracefully handle failures from external services like LLM providers without crashing.

2.3.3. Performance

- The AI analysis part of the PR analysis workflow must be completed within a maximum of 5 minutes to avoid slowing down the review cycle.
  - Issue Agent and Alignment Agent should respond within 2 minutes.
  - AI comment generation should take no more than 2 minutes.
  - PR enrichment should take no more than 5 minutes.

- The vibe reviewing phase, which depends on human collaboration, should be responsive; hence, the response time should be under 15 seconds.

2.3.4. Security

- All user source code and PR data shall be encrypted in transit.
- Mergent shall authenticate via a secure Authentication Gateway using OAuth 2.0 or JWT-based tokens.
- User source code must not be used to train general AI models without the user's explicit consent.

2.3.5. Availability

- The core service shall maintain a high uptime of at least 99% for daily development workflows.
- Shall be fault-tolerant to prevent a single agent failure from causing a system-wide outage.
- Shall have backup and disaster recovery plans to minimize downtime.

2.3.6. Scalability

- Mergent should handle at least 50 concurrent PR analyses across different repositories without degradation.
- Each AI agent shall be scaled independently to maintain performance under high load.

2.4 Pseudo Requirements

- The system must implement its version control integration specifically for the client company where Mergent will be tested.
- The system must perform all issue tracking and management operations via the specific issue and project tracking software which is used by the company where Mergent will be tested. This will ensure Mergent will align with the testing client's existing project management workflow.

2.5 System Models

2.5.1 Scenarios

2.5.1.1 Issue Linkage

- The PR author opens the pull request.
- Mergent identifies the most relevant issues related to the pull request. It is checked for the threshold of similarity.
- Mergent sees an issue that passes the threshold of the automatic linkage, and it links the issue with the pull request.

- Mergent makes a comment to the PR with a text including the linked issue and other candidate issues.

2.5.1.2 Issue Generation

- The PR author opens the pull request.
- Mergent finds the most related issues to the pull request. It is checked for the threshold of similarity.
- Mergent does not identify any issue that exceeds the threshold of automatic linkage.
- Mergent comments most related issues to the PR and is waiting for issue generation from the PR author.
- The PR author requests issue generation by commenting on the PR.
- Mergent sees the “create issue” command and creates a new issue. It links the issue to the PR by commenting on the PR.

2.5.1.3 Alignment Check

- The PR Author opens the pull request, and the issue is linked to the PR by the Mergent bot or by the PR Author.
- Mergent inspects the data of the pull request, issue, and code difference, and evaluates the alignment of the PR with its assigned issue.
- Mergent identifies the alignment as “Missing”. It labels the PR and comments on its reasoning why it’s “Missing”.
- The PR author implements the missing feature. The PR author retriggers the Mergent alignment check flow.
- Mergent reevaluates the PR and the issue alignment. Mergent identifies the PR as “Exact”. It comments on the reasoning of the decision and labels PR as “Exact.

2.5.1.4 AI Comment

- The PR Author opens the pull request.
- Mergent evaluates the code difference and writes a review that contains suggestions, bug analyses, etc. It posts a review comment on the PR.
- The PR Author adds new features and reactivates the AI Comment feature.
- Mergent reevaluates the code difference and writes a review related to the updated PR and publishes it on the PR.

2.5.1.5 PR Enrichment

- The PR Author opens the pull request.
- The reviewer starts to review the pull request.
- Mergent inspects the code difference with codebase knowledge.
- Mergent calculates a risk score and flags bug-prone areas.
- Mergent makes a comment about the risk analysis, including risk score, affected files, and bug-prone areas.

2.5.1.6 Vibe Reviewing

- The PR Author opens the pull request.
- The reviewer starts to review the pull request.
- The reviewer opens the Mergent Vibe Reviewing extension.
- The reviewer asks about a function to learn what the function is doing.
- Mergent inspects the codebase and responds back an explanation of the function.
- The reviewer continues to review. The reviewer requests a refinement from the tool.
- Mergent tool inspects and refines the review and suggests improvements.

2.5.1.7 Post-Review Feedback

- The PR Author opens a pull request.
- Reviewers review the pull request.
- The PR Author or reviewer requests post-review feedback from Mergent.
- Mergent inspects the reviews and unifies reviews in a summary. Mergent provides follow-up suggestions and key improvement areas.
- Mergent publishes the unified review on a pull request.

2.5.1.8 Organization Monitoring Dashboard

- Team Lead opens the monitoring dashboard.
- Team Lead sees different sections to view.
- Team Lead clicks the “Organization Productivity” section. The Team Lead views information related to Pull Requests, such as the average merge time of pull requests and the number of merged pull requests over the last 6 months.
- The Team Lead compares the average merge time of pull requests before and after they started using Mergent tools.
- The Team Lead clicks on another section, “Current Pull Requests”.
- Team Lead views linkages of issues and pull requests.
- The Team Lead chooses to view the timeline, where they see the current pull requests and issues in chronological order and in a calendar.

2.5.2 Use-Case Model

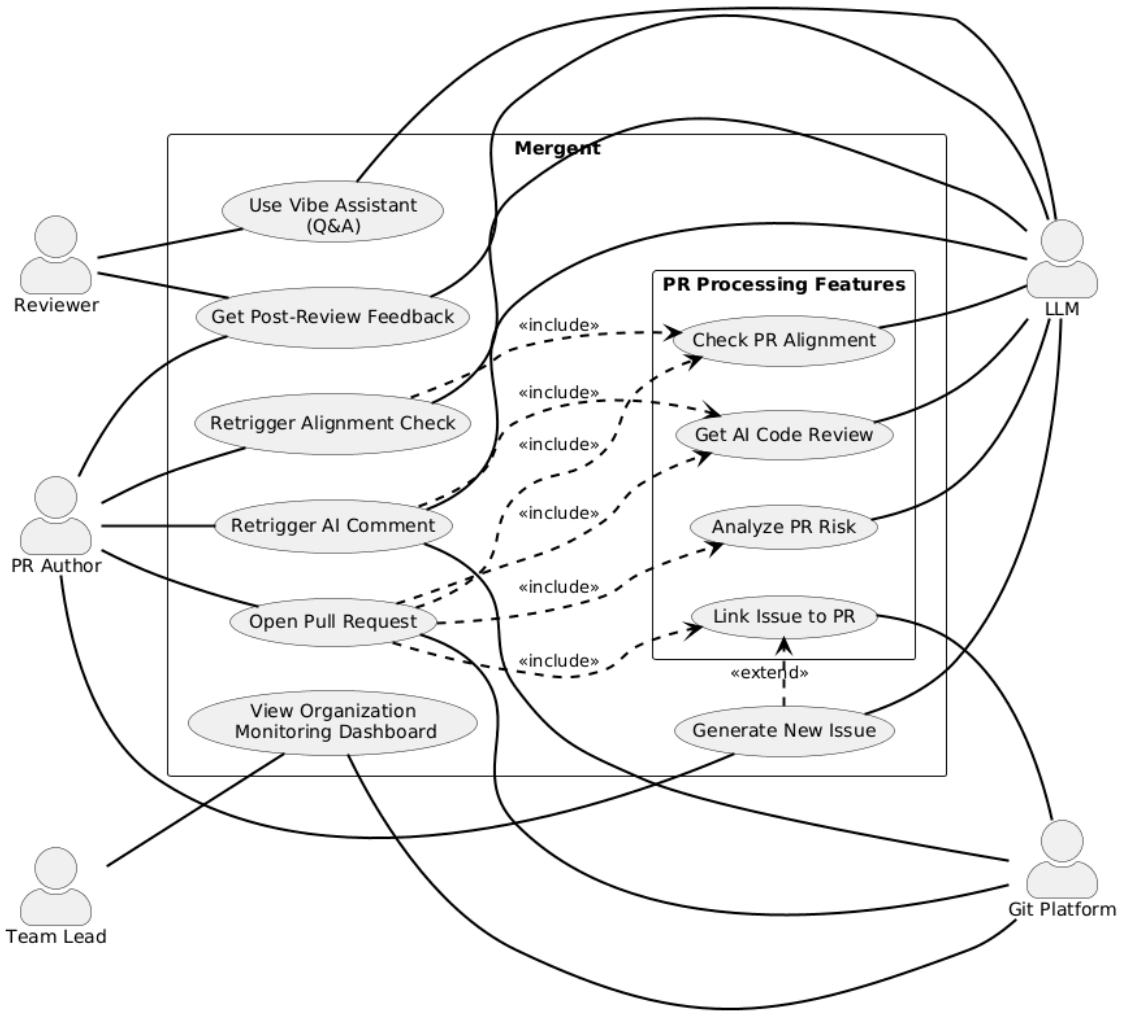


Figure 1: Use-Case Model

2.5.3 Class Model

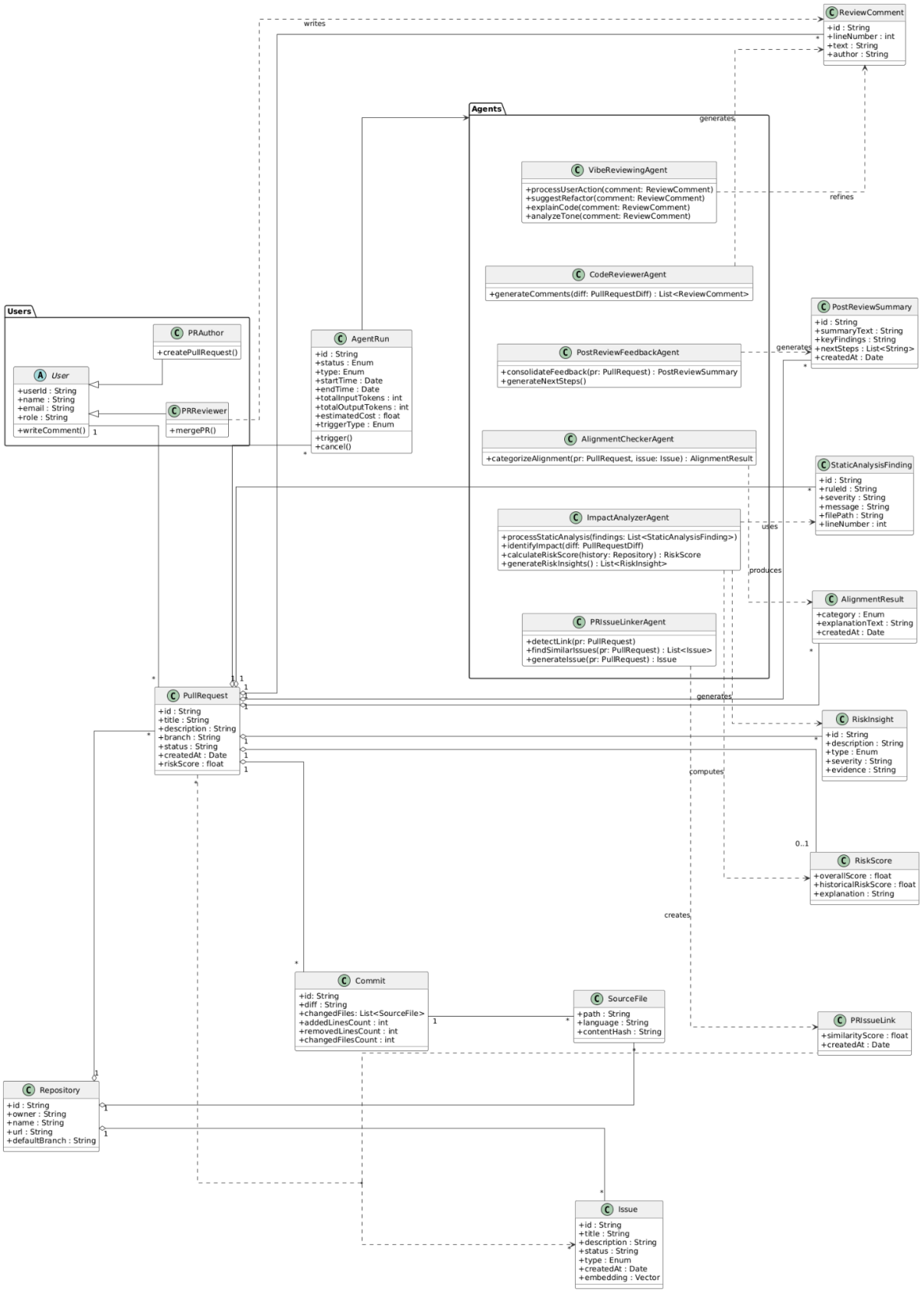


Figure 2: Class Model

2.5.4 Dynamic Models

2.5.4.1 Activity Diagram

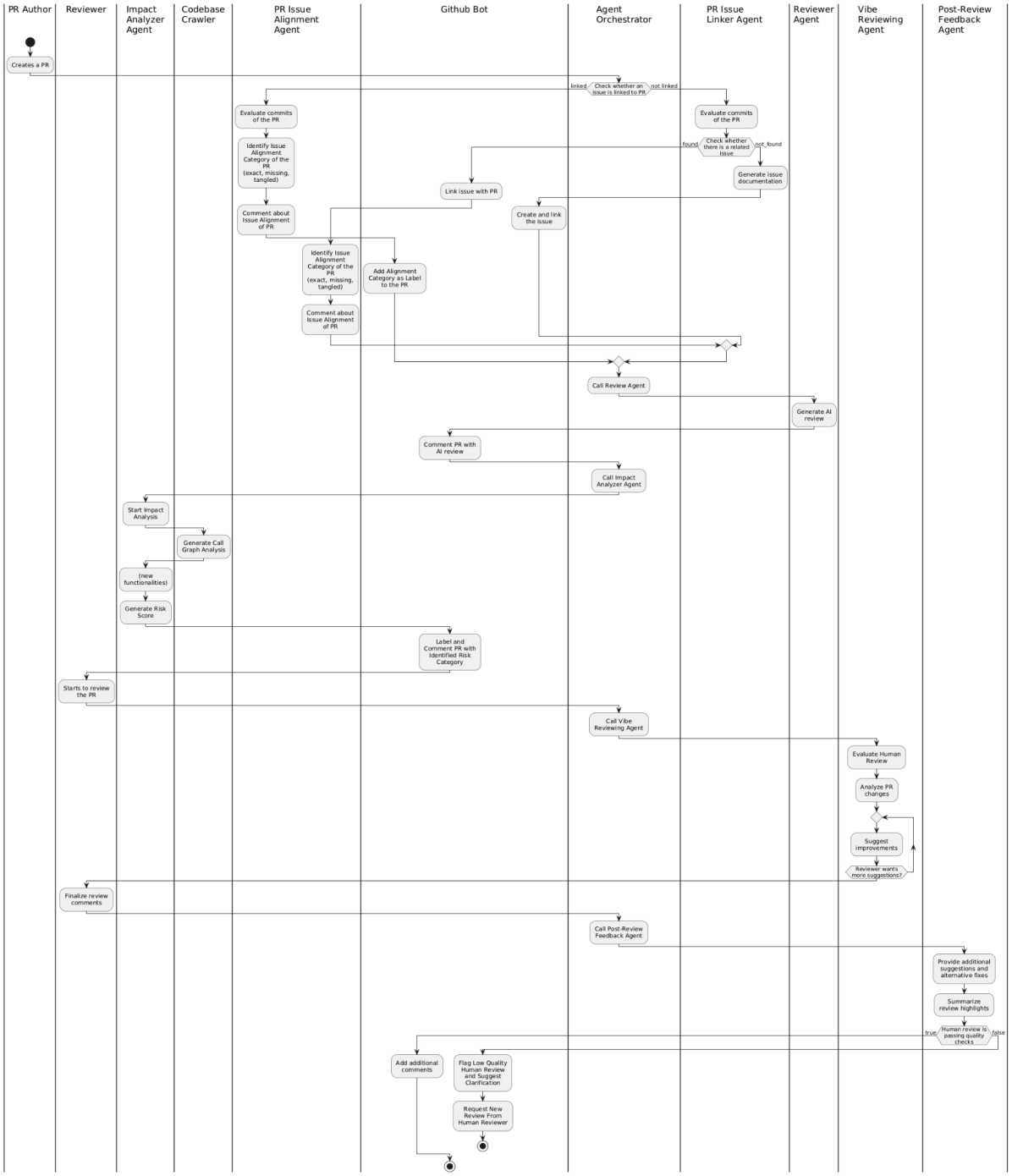


Figure 3: Activity Diagram

2.5.4.2 State Diagram

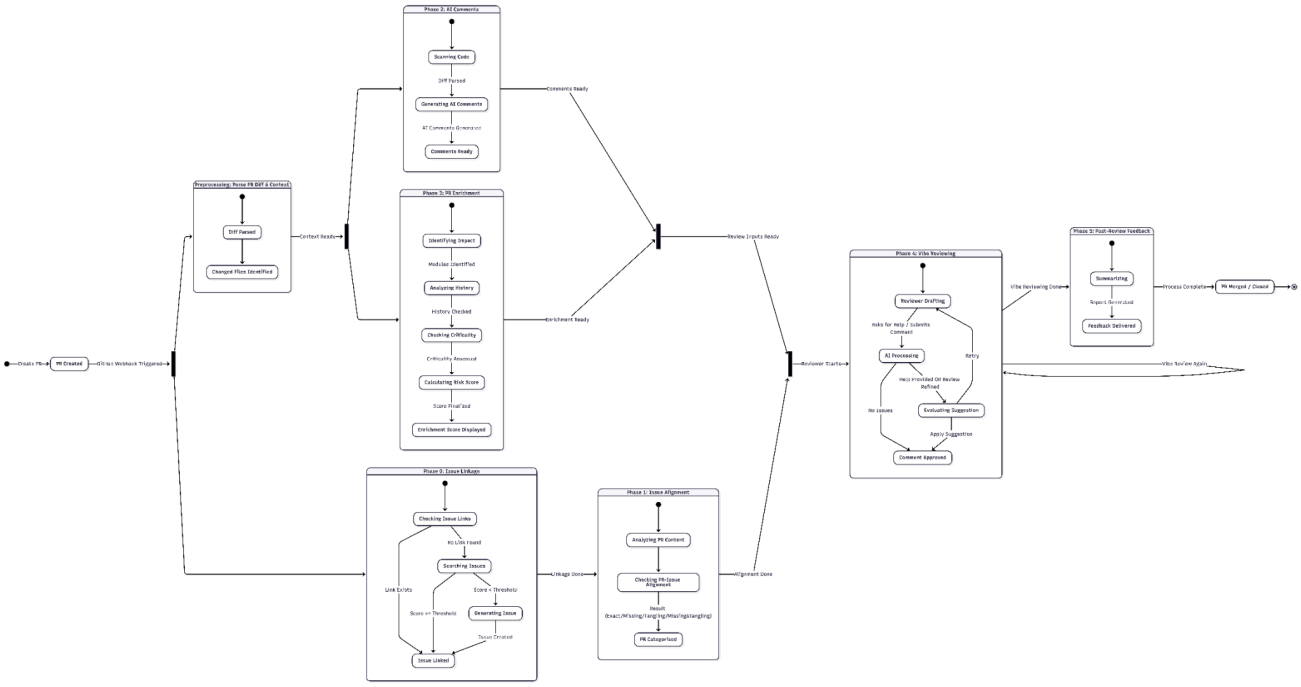


Figure 4: State Diagram

2.5.5 User Interface

2.5.5.1 Issue Linkage

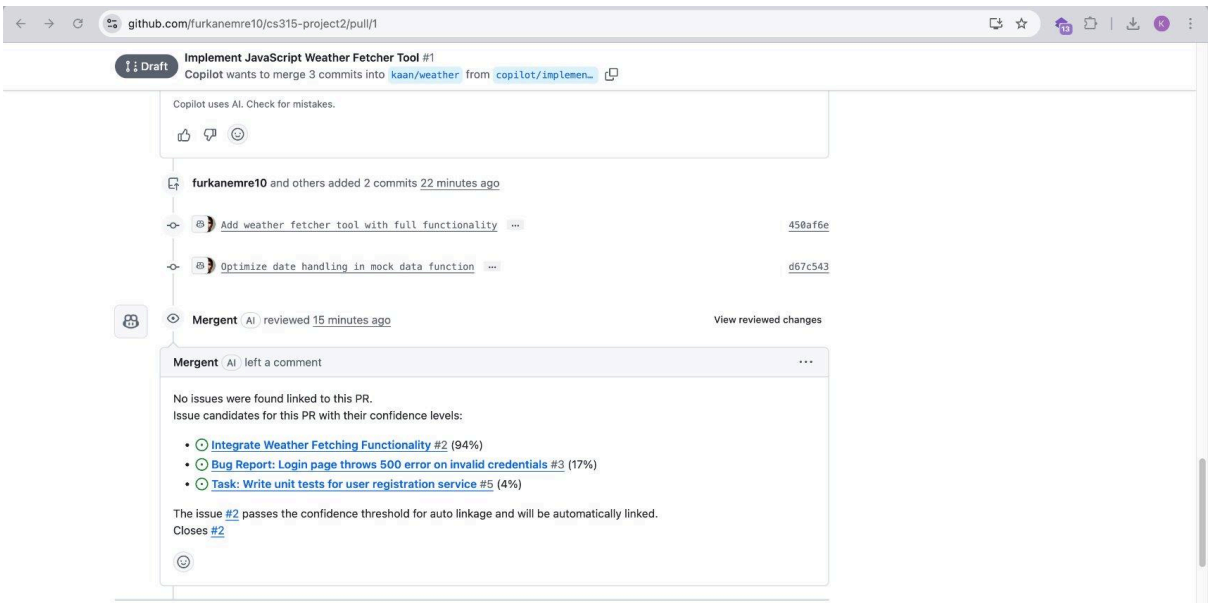


Figure 5: Issue Linkage User Interface

When a pull request is opened, Mergent automatically analyzes the PR title, description, commits, and changed files to identify relevant issues. The bot comments directly on the PR, listing candidate issues along with confidence scores

and indicating which issue passes the threshold for automatic linkage. This helps reviewers quickly understand the context of the change and ensures that the PR is correctly associated with its intended issue without requiring manual effort.

2.5.5.2 Issue Alignment

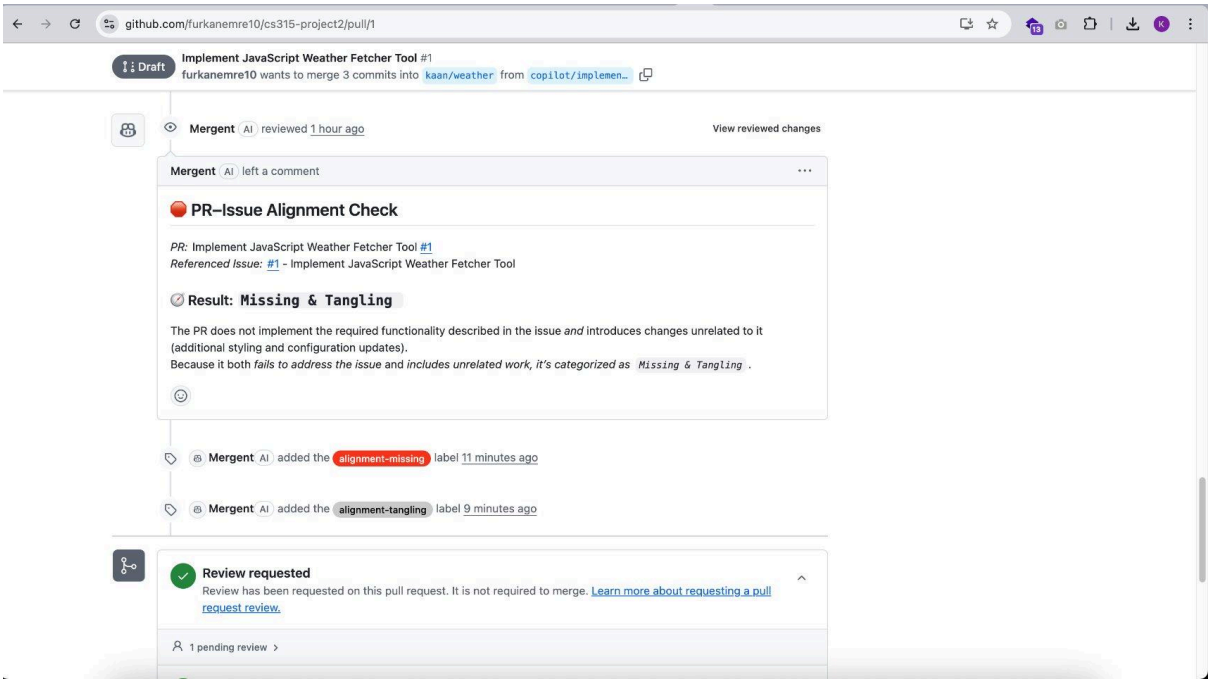


Figure 6: Issue Alignment User Interface

In the designed user interface of the issue alignment phase, the existing GitHub platform UI is used. Issue alignment is done as a GitHub comment. MergentAI gives the reasoning and identifies the alignment status of the issue and the pull request as “Missing, Tangling, Missing & Tangling or Exact”. After the identification, it is seen that a label is added with the identified alignment status.

2.5.5.3 AI Comments

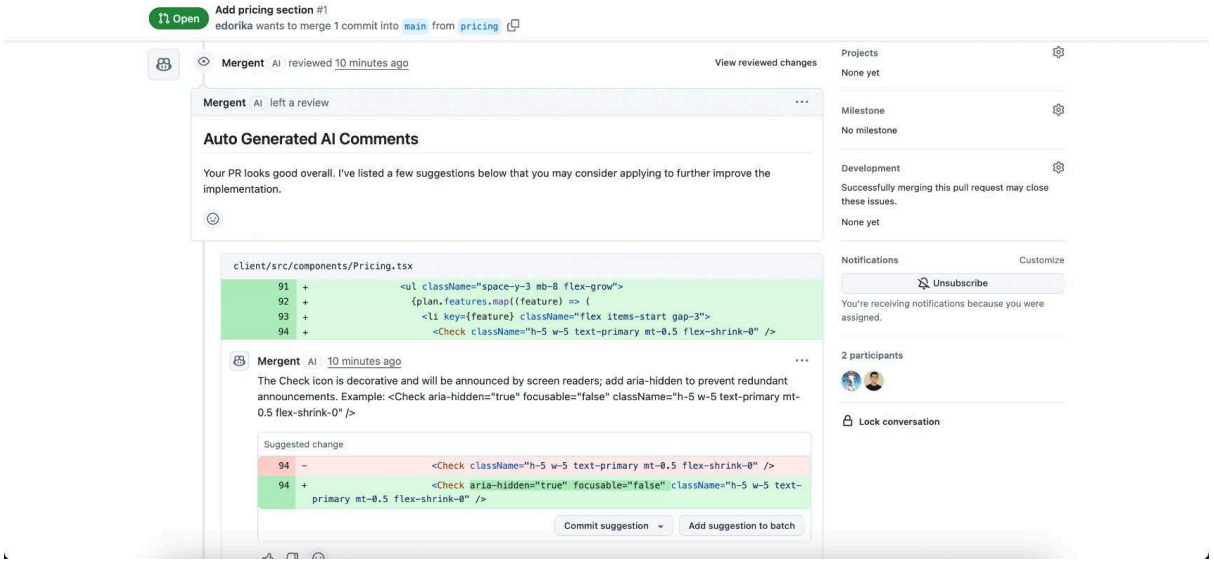


Figure 7: AI Comments User Interface

As shown in the mockup, when a pull request is reviewed, Mergent posts an automated review that includes inline, file-level comments. For each modified file, the system highlights specific lines of code and attaches contextual explanations directly below the diff. Suggested code changes are displayed using GitHub's suggestion format, allowing developers to apply fixes with a single click. This enables reviewers to quickly see which files were analyzed and what concrete improvements are recommended.

2.5.5.4 PR Enrichment

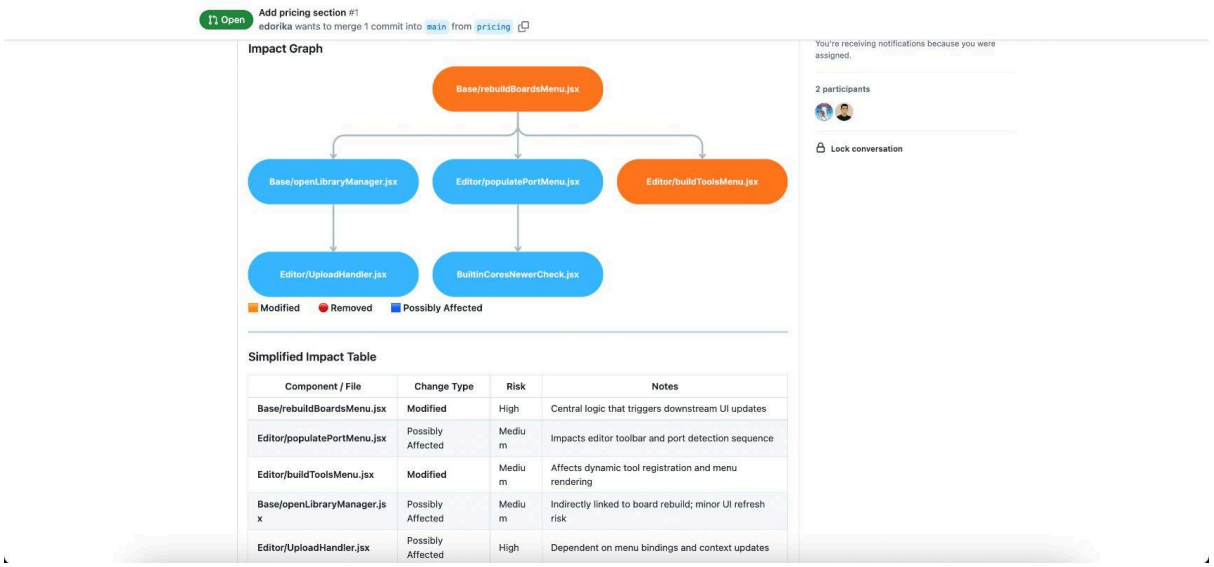
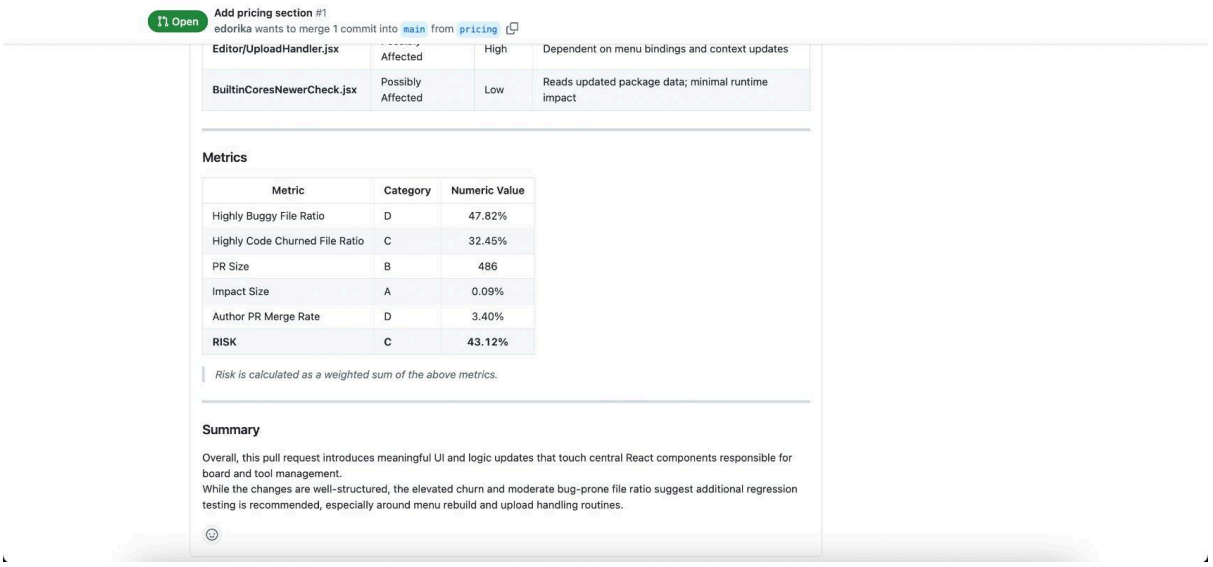


Figure 8: PR Enrichment User Interface



Open **Add pricing section** #1

edorika wants to merge 1 commit into `main` from `pricing`

Editor/UploadHandler.jsx	Affected	High	Dependent on menu bindings and context updates
BuiltinCoresNewerCheck.jsx	Possibly Affected	Low	Reads updated package data; minimal runtime impact

Metrics

Metric	Category	Numeric Value
Highly Buggy File Ratio	D	47.82%
Highly Code Churned File Ratio	C	32.45%
PR Size	B	486
Impact Size	A	0.09%
Author PR Merge Rate	D	3.40%
RISK	**C**	**43.12%**

> *Risk is calculated as a weighted sum of the above metrics.*

Summary

Overall, this pull request introduces meaningful UI and logic updates that touch central React components responsible for board and tool management.
While the changes are well-structured, the elevated churn and moderate bug-prone file ratio suggest additional regression testing is recommended, especially around menu rebuild and upload handling routines.

Figure 9: PR Enrichment User Interface

As illustrated in the mockups, Mergent enriches the pull request by visualizing the potential impact of code changes. An impact graph is displayed at the top, showing the modified file and its downstream dependencies, with color-coded nodes indicating modified and potentially affected components. Below the graph, a simplified impact table summarizes each affected file, the type of change, associated risk level, and explanatory notes. This view helps reviewers quickly understand which parts of the system may be indirectly impacted by the PR and which files require closer attention during review.

2.5.5.5 Vibe Reviewing

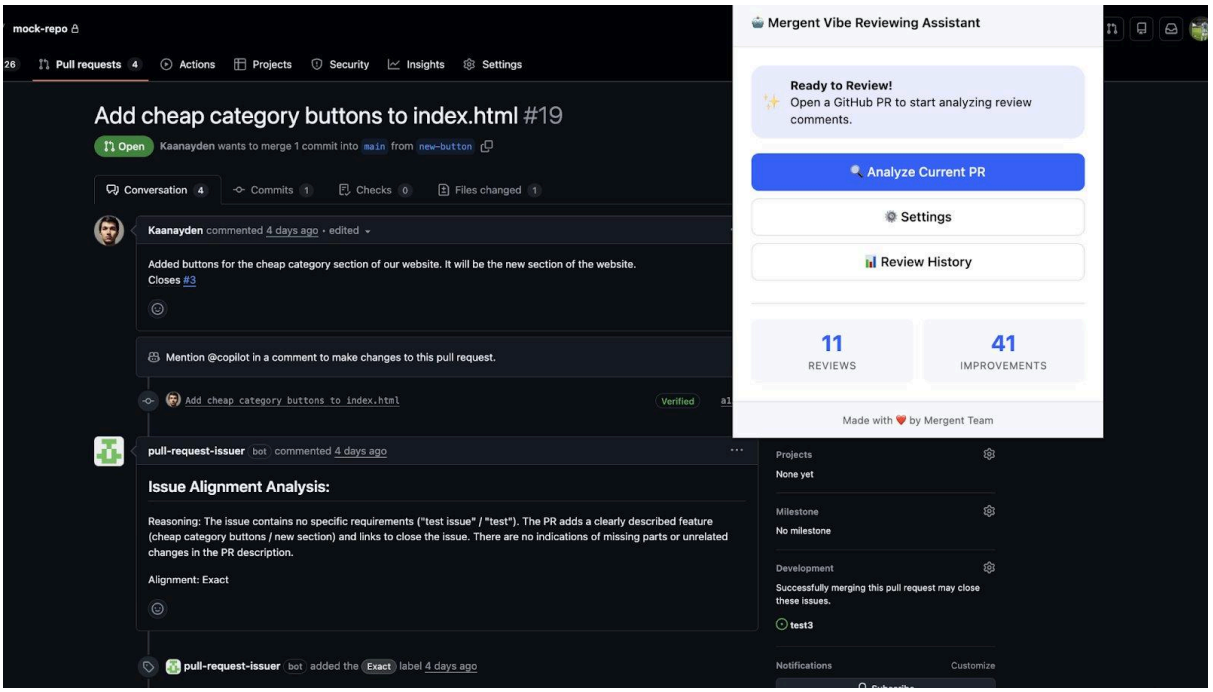


Figure 10: Vibe Reviewing User Interface

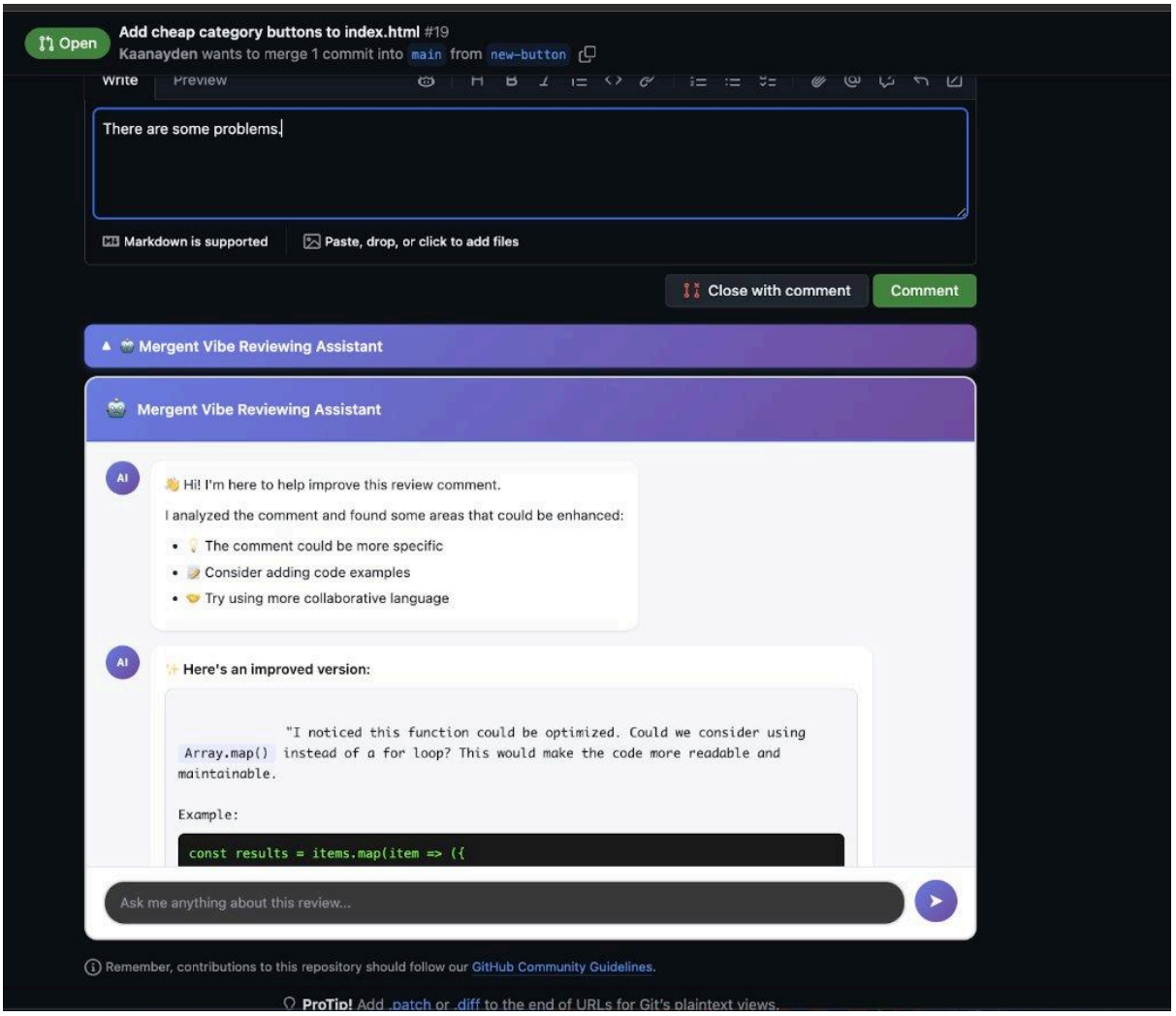


Figure 11: Vibe Reviewing User Interface

As illustrated in the mockups, Mergent supports reviewers through an interactive Vibe Reviewing extension integrated directly into the pull request interface. Chatbot dialog is injected into the GitHub page by extension. The reviewer opens the extension while reviewing the code and can ask questions about specific functions to understand their behavior. Afterward, Mergent analyzes the codebase and provides clear explanations. When the reviewer requests a refinement, the extension suggests an improved version of the review comment, which the reviewer can accept or further refine through continued interaction. A lightweight side panel displays the number of reviews and improvements made using the extension so far, allowing the reviewer to track their review activity and continue the review without leaving the pull request context.

2.5.5.6 Post-Review Feedback

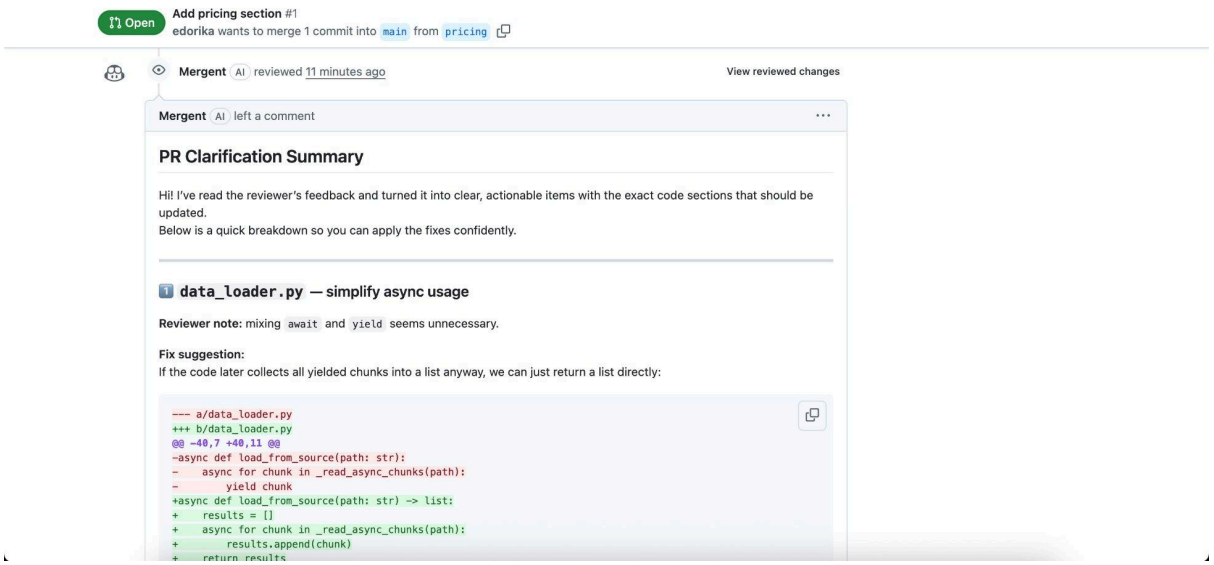


Figure 12: Post-Review Feedback User Interface

As shown in the mockup, after the review process is completed, Mergent posts a consolidated PR clarification summary directly on the pull request. The summary groups reviewer comments by file and topic, transforming individual review notes into clear, actionable items. For each item, the system highlights the relevant code section and presents concrete fix suggestions, allowing the PR author to quickly understand what needs to be changed and why. This final report helps reduce confusion and minimizes additional back-and-forth between reviewers and authors.

3 Other Analysis Elements

3.1 Consideration of Various Factors in Engineering Design

The design of Mergent is influenced primarily by economic and environmental factors due to its dependency on Large Language Models (LLMs), cloud storage, and third-party platform APIs. LLM inference introduces a direct economic cost, which affects development decisions such as model selection, caching strategies, and frequency of analysis. In addition to inference cost, storing repository data, embeddings, and metadata requires cloud storage, while running the system continuously in production requires server resources. These add to the overall operational expenses. Efficient use of cloud resources and API calls is therefore necessary to keep the system financially sustainable. Related to this, the computational needs of LLM calls and server usage introduce an environmental impact, since large-scale model usage consumes energy. This motivates design

choices such as minimizing unnecessary calls, reducing storage overhead, and prioritizing efficient workflows.

Other factors, such as global and social aspects, have a minor effect on the analysis. Mergent supports collaborative software development workflows, which are used in diverse organizations, but its functionality does not depend on specific cultural practices or local norms. Likewise, the system does not influence public health or safety, nor does it introduce risks to public welfare, as it operates strictly within software development environments.

Table 1: Various Factors in Engineering Design

	Effect level	Effect
Public health	0	-
Public safety	0	-
Public welfare	0	-
Global factors	2	Used in global teams, minor influence on design.
Cultural factors	1	Review practices vary, effect is minimal.
Social factors	2	Supports collaboration but does not depend on it.
Environmental factors	4	LLM usage increases energy consumption.
Economic factors	8	LLM/API server costs shape design and frequency of use.

3.1.1 Constraints

Implementation Constraints

- **Availability of suitable training data**: Limited training data for pull requests, issues, and review comments reduces the system's ability to generalize across different projects.
- **Reliance on LLM APIs:** Mergent's performance depends on LLM quality and external API access rather than local compute power.

- **Integration with external platforms:** GitHub or JIRA APIs have rate limits and restricted control over the review process, which limits functionality.
- **Limited control over workflows**: Since Mergent works within existing development environments and version control systems, it can only extend workflows through APIs or bots, not modify them directly.

Economic Constraints

- **Cost of LLM API calls:** Frequent use of LLMs for code analysis and feedback generation increases operational expenses.
- **Infrastructure and hosting costs:** Cloud hosting, API integrations, and deployment infrastructure contribute to budget limitations.

Sustainability Constraints

- **High energy and resource usage:** Frequent LLM calls and large-scale analyses can increase energy consumption and operational costs.
- **Scalability limitations:** The system must remain lightweight to avoid performance degradation and excessive infrastructure demand as usage grows.

Time Constraints

- **Limited development time:** As a two-semester capstone project, development time is finite.
- **Competing project activities:** Research, testing, and documentation also require time, restricting the implementation of additional features and integrations beyond the core scope.

Privacy Constraints

- **Handling of sensitive information:** Mergent processes real code, commit messages, comments, issues, and pull requests that may contain proprietary data, requiring strict confidentiality.
- **External API interactions:** Data passed to external services must be protected to prevent leaks outside the authorized environment.

3.1.2 Standards

The project follows UML 2.5.1 for system and process modeling to maintain consistency and clarity in design diagrams [3]. IEEE 830-1998 standards are applied for requirements specification to ensure structured and well-documented system

requirements [4]. In addition, general IEEE software engineering practices are followed to support maintainability, reliability, and quality throughout development.

3.2 Risks and Alternatives

Risk: Azure free LLM credits are finished

Mergent's current workflow relies heavily on the free quota of LLM credits provided by Azure. Once these credits are exhausted, most of the agents and chain-of-thought operations will stop working because the inference requests cannot be served. This would have a critical impact on the system, as all intelligent phases (alignment, enrichment, vibe reviewing) depend on LLM output. The alternative plan is to access more credits from Azure, evaluate free or academic credit options from other APIs, request sponsored credits, or temporarily use personal funding while exploring limited-call or local model configurations to reduce cost.

Risk: Switching from GitHub to Bitbucket (industry partner)

Although Mergent is developed and tested on GitHub, the external company may use Bitbucket or a self-hosted GitLab instance. API structures, authentication flows, and webhook events differ significantly between these platforms, which may break parts of the existing workflow if a direct switch is required. Adapting the system to a new environment would introduce delays and require changes in authentication, PR data extraction, and event handling. The alternative plan is to develop abstract interfaces and generic connectors so that changing the backend platform only requires switching the implementation layer without rewriting core logic.

Risk: Not finding enough real-world data for evaluation

For some phases, especially PR enrichment and reviewer assistance, it may be difficult to find high-quality, labeled datasets to evaluate agent performance. While issue linkage and alignment can be tested using the literature datasets, other phases do not have existing benchmarks and require realistic PR–issue pairs. Without proper testing data, fine-tuning prompts, measuring accuracy, and iterating on the workflow become difficult, reducing the credibility of results. The alternative plan is to collect new datasets by mining open-source repositories, extracting real pull requests, and building manual evaluation datasets with annotations.

Risk: Delays in implementation (especially PR enrichment)

The PR enrichment phase has multiple planned features, including risk scoring, impact graphs, recurring issue detection, and reviewer insights.

Implementing all of these within a limited timeline may not be possible if unexpected issues arise in earlier phases. This risk is moderate, as some features may take longer than planned to integrate with real data. The alternative plan is to prioritize the core features with the highest impact and drop or postpone the less critical features if delays occur.

Risk: Integration problems with the external company

The external testing partner may not allow cloud-based LLMs due to internal security policies, or they may require all model execution to happen on local/on-premise infrastructure. In this case, Mergent's existing pipeline may not run because small local models usually do not perform at the same level as cloud LLMs. Integrating into this environment requires exploring local models and optimizing prompts for smaller architectures. The alternative plan is to switch to local or hybrid LLMs and adjust the evaluation criteria to match realistic performance expectations.

Table 2: Risks

	Likelihood	Effect on the project	B Plan Summary
Azure free credits finished	High	High	Use other APIs, request credits, reduce calls
Switch to Bitbucket	High	Medium/High	Write abstract interfaces and connectors
Not finding enough real-world data for evaluation	Medium	Medium	Use intuition or developer feedback for evaluation.
Delays in implementation (especially PR enrichment)	Low	Low	Enhance parallelism or use low-latency api providers.

Integration problems with the external company	Low	High	Try to enhance the current system with open-source model alternatives

3.3 Project Plan

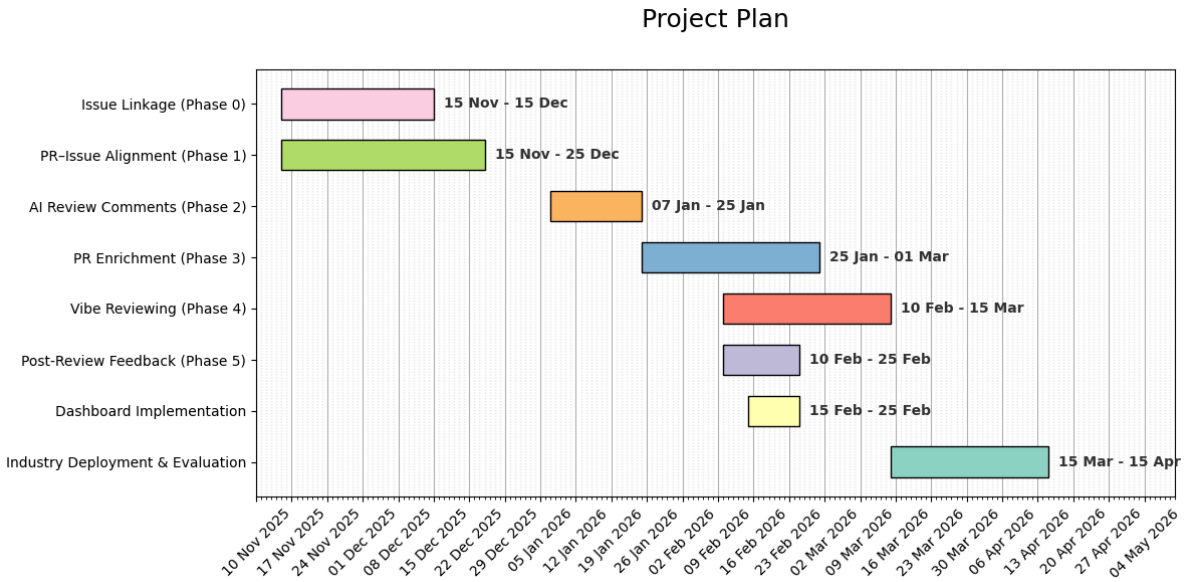


Figure 13: Gannt Chart of Project Plan

WP#	Work package title	Leader	Members involved
WP1	Issue Linkage (Phase 0)	Furkan Emre Kolukırık	Erdem Aydın Hasan Kutluhan Şıpka
WP2	PR–Issue Alignment (Phase 1)	Oğuzhan Genç	Kaan Aydeniz Erdem Aydın
WP3	AI Review Comments (Phase 2)	Kaan Aydeniz	Oğuzhan Genç Furkan Emre Kolukırık
WP4	PR Enrichment (Phase 3)	Furkan Emre Kolukırık	Kaan Aydeniz Hasan Kutluhan Şıpka

WP5	Vibe Reviewing (Phase 4)	Hasan Kutluhan Şıpka	Furkan Emre Kolukırık Kaan Aydeniz
WP6	Post-Review Author Support (Phase 5)	Erdem Aydın	Hasan Kutluhan Şıpka Oğuzhan Genç
WP7	Detailed Design Report	None	Furkan Emre Kolukırık Kaan Aydeniz Hasan Kutluhan Şıpka Oğuzhan Genç Erdem Aydın
WP8	Final Report	None	Furkan Emre Kolukırık Kaan Aydeniz Hasan Kutluhan Şıpka Oğuzhan Genç Erdem Aydın
WP9	Demo Preparation (Working Prototype)	None	Furkan Emre Kolukırık Kaan Aydeniz Hasan Kutluhan Şıpka Oğuzhan Genç Erdem Aydın
WP10	Dashboard Implementation	Kaan Aydeniz	Furkan Emre Kolukırık Oğuzhan Genç
WP11	Industry Deployment & Evaluation	None	All Members

WP 1: *Issue Linkage (Phase 0)*			
Start date: *15.11.2025* **End date:** *15.12.2025*			
Leader:	Furkan Emre Kolukırık	**Members involved:**	Erdem Aydın Hasan Kutluhan Şıpka
Objectives: Automatically detect or generate links between pull requests and issues by using NLP, retrieval, and heuristic signals.			

Tasks:
Task 1.1 **Research:** study classical IR, ReLink (TF-IDF + Word2Vec), heuristic linkage methods.

Task 1.2 **Implementation:** implement candidate linking agent with scoring (semantic + heuristic features).

Task 1.3 **Dataset:** collect PR–issue pairs from open-source repos; clean and normalize the data.

Task 1.4 **Testing & Improvement:** evaluate accuracy (MRR, Recall@k), refine scoring logic.

Deliverables
Working linkage agent

Evaluation metrics and report

Dataset description

WP 2: *PR–Issue Alignment (Phase 1)*			
Start date: *15.11.2025* **End date:** *25.12.2025*			
Leader:	Oğuzhan Genç	**Members involved:**	Kaan Aydeniz Erdem Aydın

Objectives:

Verify whether a PR correctly addresses its linked issue using semantic analysis and target mismatch detection.

Tasks:
Task 2.1 **Research:** analyze alignment approaches, PR summarization techniques, multi-step prompting, and example outputs from tools like CodeRabbit and DeputyDev.

Task 2.2 **Implementation:** implement alignment classifier using LLM reasoning or lightweight heuristics.

Task 2.3 **Dataset:** extract examples of aligned/misaligned PR–issue pairs; label data if necessary.

Task 2.4 **Testing & Improvement:** measure accuracy and refine model prompts and decision thresholds.
Deliverables *Alignment agent* *Alignment evaluation report* *Alignment dataset*

WP 3: *AI Review Comments (Phase 2)*			
Start date: *07.01.2026* **End date:** *25.01.2026*			
Leader:	Kaan Aydeniz	**Members involved:**	Oğuzhan Genç Furkan Emre Kolukırık
Objectives: *Generate initial AI-assisted review comments to support reviewers in identifying issues in the PR.*			
Tasks: ***Task 2.1*** **Research:** examine AI code review literature, prompt engineering strategies, and structured comment formats. ***Task 2.2*** **Implementation:** build a review agent that suggests actionable comments and identifies code issues. ***Task 2.3*** **Dataset:** collect real review comments from repositories like TensorFlow, Kubernetes, or similar OSS projects. ***Task 2.4*** **Testing & Improvement:** check usefulness and clarity of comments; refine tone, format, and specificity.			
Deliverables *Review comment agent* *Prompt design document* *Evaluation samples*			

WP 4: *PR Enrichment (Phase 3)*			
Start date: *25.01.2026* **End date:** *01.03.2026*			
Leader:	Furkan Emre Kolukırık	**Members involved:**	Kaan Aydeniz Hasan Kutluhan Şıpka
Objectives: *Provide deeper contextual signals (risk scoring, recurring problem areas, change-impact hints) to reviewers.*			
Tasks: ***Task 2.1* Research:** investigate CHID impact scoring and graph-based analysis. ***Task 2.2 I*mplementation:** combine risk analytics (history) and structural analysis (impact graph) into enrichment output. ***Task 2.3* Dataset:** mine repositories for commit history, bug-fix patterns, and PR metadata. ***Task 2.4* Testing & Improvement:** check if enrichment context helps reviewers identify risky changes; refine scoring.			
Deliverables *Enrichment module (risk + impact)* *Risk evaluation logs* *Visualization mockups*			

WP 5: *Vibe Reviewing (Phase 4)*			
Start date: *10.02.2026* **End date:** *15.03.2026*			
Leader:	Hasan Kutluhan Şıpka	**Members involved:**	Furkan Emre Kolukırık Kaan Aydeniz
Objectives: *Assist human reviewers by evaluating comment clarity and helping them produce higher-quality feedback.*			

Tasks: ***Task 2.1*** **Research:** explore large language models for feedback quality, vague comment detection, and suggestion rewriting. ***Task 2.2*** **Implementation:** build reviewer coach agent detecting vague comments and suggesting specific feedback. ***Task 2.3*** **Dataset:** manually collect reviewer comments labeled as vague vs. specific from real PRs. ***Task 2.4*** **Testing & Improvement:** evaluate improvement by comparing before/after comments; refine templates.
Deliverables *Reviewer coach agent* *Comment quality evaluation* *Short demo showcasing "vague → specific" rewrite*

WP 6: *Post-Review Feedback (Phase 5)*			
Start date: *10.02.2026* **End date:** *25.02.2026*			
Leader:	Erdem Aydın	**Members involved:**	Hasan Kutluhan Şıpka Oğuzhan Genç
Objectives: *Summarize the PR's review outcome for the author, highlighting main points and suggested next steps.*			
Tasks: ***Task 2.1*** **Research:** analyze summarization techniques for multi-turn discussion and identifying core review signals. ***Task 2.2*** **Implementation:** implement summarization agent: extract major review points, suggested tests, and closing summary. ***Task 2.3*** **Dataset:** collect PR discussions and verified summaries; prepare small evaluation set.			

Task 2.4 **Testing & Improvement:** check summary quality with metrics (e.g., coherence, coverage); refine prompt patterns.
Deliverables *Author support agent* *Summary generation examples* *Summary evaluation report*

WP 7: *Detailed Design Report*			
Start date: *TBA* **End date:** *TBA*			
Leader:	None	**Members involved:**	All Members
Objectives: *Write the report*			
Tasks: ***Task 2.1*** **Analyze:** analyze report requirements and context, divide jobs among members ***Task 2.2*** **Writing:** write the report and merge each member's part.			
Deliverables *Analysis & Requirements Report*			

WP 8: *Final Report*			
Start date: *TBA* **End date:** *TBA*			
Leader:	None	**Members involved:**	All Members
Objectives:			

Write the report
Tasks: ***Task 2.1*** **Analyze:** analyze report requirements and context, divide jobs among members ***Task 2.2*** **Writing:** write the report and merge each member's part.
Deliverables *Final Report*

WP 9: *Demo Preparation (Working Prototype)*			
Start date: *10.12.2025* **End date:** *22.12.2025*			
Leader:	None	**Members involved:**	All Members
Objectives: Get prepared for the demo presentation. Create some use cases and scenarios to illustrate for the jury members.			
Tasks: ***Task 2.1*** **Presentation:** prepare a presentation that illustrates the latest version of the project ***Task 2.2*** **Preparation:** divide the presentation parts among members and try some mockups before the actual presentation to the jury members			
Deliverables *Demo presentation*			

WP 10: *Dashboard Implementation*
Start date: *15.02.2025* **End date:** *25.02.2025*

Leader:	Kaan Aydeniz	**Members involved:**	Furkan Emre Kolukırık Oğuzhan Genç
Objectives: Provide repository-level visibility into how Mergent is used and how it affects the PR review process by presenting meaningful statistics and performance metrics.			
Tasks: **Research:** investigate software analytics dashboards, DevOps metrics (DORA metrics), PR lifecycle statistics, and cost-aware AI monitoring practices. Identify which metrics are meaningful for PR reviews and AI-assisted workflows. **Implementation:** design and implement a dashboard that visualizes key statistics such as number of analyzed PRs, number of AI-assisted reviews, number of comments generated, average review duration per PR, and average LLM cost per review.			
Deliverables: ● *Interactive analytics dashboard* ● *Metric definitions and documentation* ● *Sample repository analytics report*			

WP 10: *Industry Deployment & Evaluation*			
Start date: *15.03.2026* **End date:** *15.04.2026*			
Leader:	None	**Members involved:**	All Members
Objectives:			

Evaluate Mergent in a real industry environment by integrating it into a company repository and observing its behavior under realistic development workflows.
Tasks: ● **Research:** analyze the company's development environment, repository platform (e.g., GitHub, Bitbucket), security constraints, and review workflow to understand integration requirements. ● **Implementation:** deploy Mergent to the company's repository using the appropriate platform APIs, configure authentication, webhooks, and access permissions, and adapt workflows if needed. ● **Dataset:** collect real PR data, review comments, timestamps, and usage logs generated during the testing period while respecting confidentiality constraints. ● **Testing & Improvement:** observe system performance, review accuracy, latency, and usability; gather qualitative feedback from developers and adjust prompts, thresholds, or workflows accordingly.
Deliverables: ● *Deployed version of Mergent in an industry repository* ● *Industry evaluation report (qualitative + quantitative)* ● *Feedback summary and improvement notes*

3.4 Ensuring Proper Teamwork

Our team follows a flexible, mixed-role approach, where everyone contributes to whatever is needed at the moment, whether it's development, design, research, or

documentation. Instead of fixed roles, we organize our work through weekly task planning, which clearly assigns responsibilities and ensures steady progress.

All coordination and documentation are centralized in Notion, where we track tasks, meeting notes, and shared resources. We also conduct weekly team meetings to review progress, resolve blockers, and realign tasks when needed.

This system keeps us organized, transparent, and collaborative while remaining adaptable.

3.5 Ethics and Professional Responsibilities

The development of this project requires adherence to professional and ethical conduct within the team and in interactions with external collaborators. All team members are expected to contribute responsibly, avoid plagiarism in both code and documentation, and ensure that all work produced is original and properly attributed. Tasks and responsibilities of the project should be distributed fairly to the Mergent team to maintain a balanced and respectful working environment.

Since the project will be tested or demonstrated in collaboration with an external company, confidentiality must be strictly maintained. Any proprietary information, internal documentation, or source code provided by the partner organization must not be shared outside the project group. All exchanges must comply with the company's security and privacy policies, and the ownership of both the project code and any shared assets must be clearly respected.

Professionalism also requires transparent communication, respectful teamwork, and adherence to deadlines. Ethical awareness throughout the project ensures that the development process remains fair, trustworthy, and aligned with academic and industry standards.

3.6 Planning for New Knowledge and Learning Strategies

The development of Mergent requires acquiring new technical knowledge beyond the team's existing skill set. In particular, the project depends on understanding Large Language Models (LLMs), prompt engineering, and multi-agent reasoning workflows to support the system's intelligent functionalities. Additional learning is needed for integrating external services such as GitHub APIs, storing repository metadata, and working with cloud-based infrastructures for evaluation. The team will also study techniques in software repository mining, including extracting PR–issue data, analyzing commit history patterns, and constructing small evaluation datasets.

To support this learning process, the team will adopt a combination of learning strategies. These include online tutorials, research publications in software engineering and NLP, and hands-on experimentation through rapid prototyping. New

knowledge will be shared within the team using pair programming and internal presentations so that all members gain a similar understanding of critical components. When necessary, the team will consult experts from the industry partner or seek guidance from academic literature to validate design choices. This process ensures continuous skill development throughout the project and helps refine the system design based on newly acquired knowledge.

Glossary

The following terms and abbreviations are used throughout this report to describe the technical and functional components of the Mergent system:

- AI (Artificial Intelligence): The simulation of human intelligence processes by computer systems, used in this project to automate complex code review tasks.
- Bitbucket: A Git-based source code repository hosting service that serves as one of the target integration platforms for the project.
- Code Diff: The specific set of added, removed, or modified lines of code within a pull request.
- GitHub: A developer platform that allows developers to create, store, manage, and share their code; Mergent interacts with this via its API and specialized bots.
- Human-in-the-Loop: A design strategy where the AI system serves as a collaborative partner to human reviewers to improve feedback accuracy and tone.
- Issue: A task explanation, bug report, or feature request that describes the work required for a project.
- JIRA: A proprietary issue-tracking and project management tool used to align with existing client workflows.
- JWT (JSON Web Token): A secure token format standard used for the authentication and authorization of users within the system.
- LLM (Large Language Model): Advanced AI models trained on vast amounts of data, used by Mergent to interpret code context and generate natural language reviews.
- OAuth 2.0: A standard protocol for secure user authentication and access delegation.
- PR (Pull Request): A request made by a developer to merge code changes from one branch into another, which triggers the Mergent review pipeline.
- RAG (Retrieval-Augmented Generation): A technique that provides the LLM with real-time, rule-based context from static analysis or code crawlers without requiring expensive retraining.
- Repository (Repo): The project's source code is maintained through a version control system.
- Static Analysis: Automated code inspection that identifies errors or logical issues without executing the program.
- Tangling: A state where a pull request includes additional, unrelated changes that were not mentioned in the original issue description.
- Vibe Reviewing: A human-AI partnership phase where the system refines the tone and clarity of human reviewer comments.

References

[1] U. Cihan et al., "Automated Code Review in Practice," 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), Ottawa, ON, Canada, 2025, pp. 425-436, doi: 10.1109/ICSE-SEIP66354.2025.00043.

[2] C. Liu, H. Y. Lin and P. Thongtanunam, "Too Noisy To Learn: Enhancing Data Quality for Code Review Comment Generation," 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), Ottawa, ON, Canada, 2025, pp. 236-248, doi: 10.1109/MSR66628.2025.00043.

[3] OMG, Unified Modeling Language (UML), Version 2.5.1. Needham, MA, USA: Object Management Group, Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[4] "IEEE Recommended Practice for Software Requirements Specifications," in IEEE Std 830-1998 , vol., no., pp.1-40, 20 Oct. 1998, doi: 10.1109/IEEESTD.1998.88286.

[5] A. Yaşa et al., "Evaluating ReLink for Traceability Link Recovery in Practice," 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), Montreal, QC, Canada, 2025, pp. 80-90, doi: 10.1109/SANER64311.2025.00016.

[6] A. T. Işık, H. Kübra Çağlar and E. Tüzün, "Enhancing Pull Request Reviews: Leveraging Large Language Models to Detect Inconsistencies Between Issues and Pull Requests," 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), Ottawa, ON, Canada, 2025, pp. 168-178, doi: 10.1109/Forge66646.2025.00027.

[7] I. S. Göçmen, A. S. Cezayir, and E. Tüzün, ‘Enhanced code reviews using pull request based change impact analysis’, Empir. Softw. Eng., vol. 30, no. 3, May 2025.